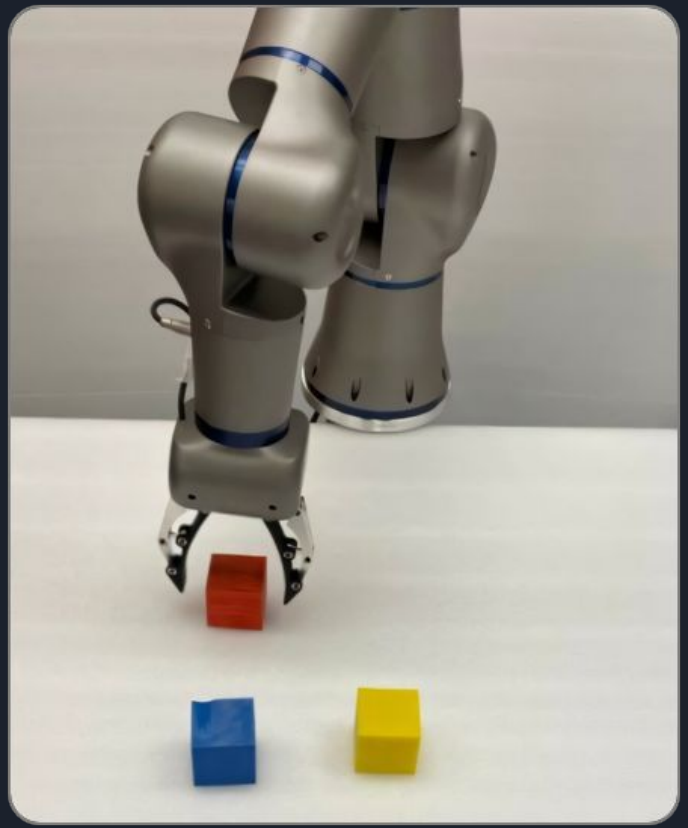
Embodied Agentic Artificial Intelligence

12.03.2026 Kajetan Rachwał

Embodied AI



Embodied AI

“Embodiment can be understood as the capacity of a cyber-physical system to process information about its physical component [...] but also about the environment in which it operates. It is thus better viewed not as an intrinsic property of the agent, but as a trait emerging from the dynamic coupling between the agent and its environment.”

Towards Embodied Agentic AI: Review and Classification of LLM- and VLM-Driven Robot Autonomy and Interaction

S. Salimpour et. al

Agenda

1. Purpose
2. Introduction to ROS2
3. RAI System Architecture
4. Use Case Walk Through
5. RAI in Multi-Agent Context

Purpose

- Leveraging LLMs, VLMs and VLAs as reasoning engines
- Human-in-the-Loop
- Native embodiment capabilities
- Integration with robotic stacks
  - ROS2 - the common standard
  - Device Integrations
- Providing guardrails and monitoring

Introduction to ROS2

<u>What is a robot?</u>

- A physical machine
- Perception capabilities
- Reasoning
- Acting on the world

Is every robot an Agent?

Robots as Embodied Agents

- Physical constraints
- Latency in changing environment
- Sensors - real data is noisy, partial, ambiguous
  - Cameras
  - Microphones
  - LIDAR
- Actuators - movement is slow, continuous, irreversible
  - Wheels
  - Grippers
  - Speakers

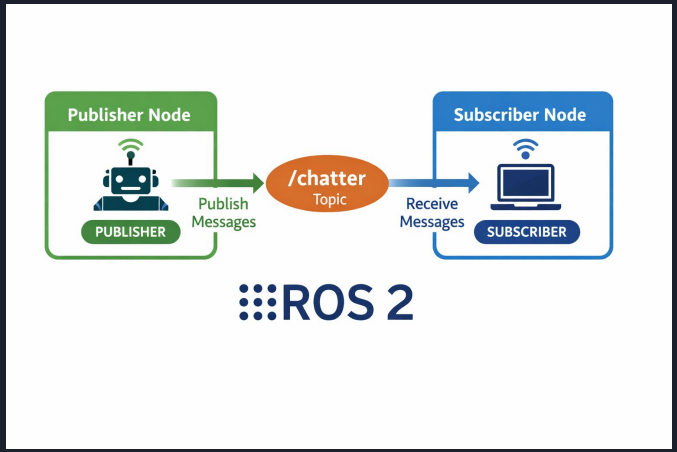
Introduction to ROS2

- Robot Operating System (not an OS, but a middleware)
- Provides standards for
  - Messaging
  - Control
  - Perception
- Based on communicating nodes
  - Topics → continuous data streams (e.g., camera feed)
  - Services → one-time requests (e.g., "open gripper")
  - Actions → long-running tasks (e.g., "navigate to kitchen")

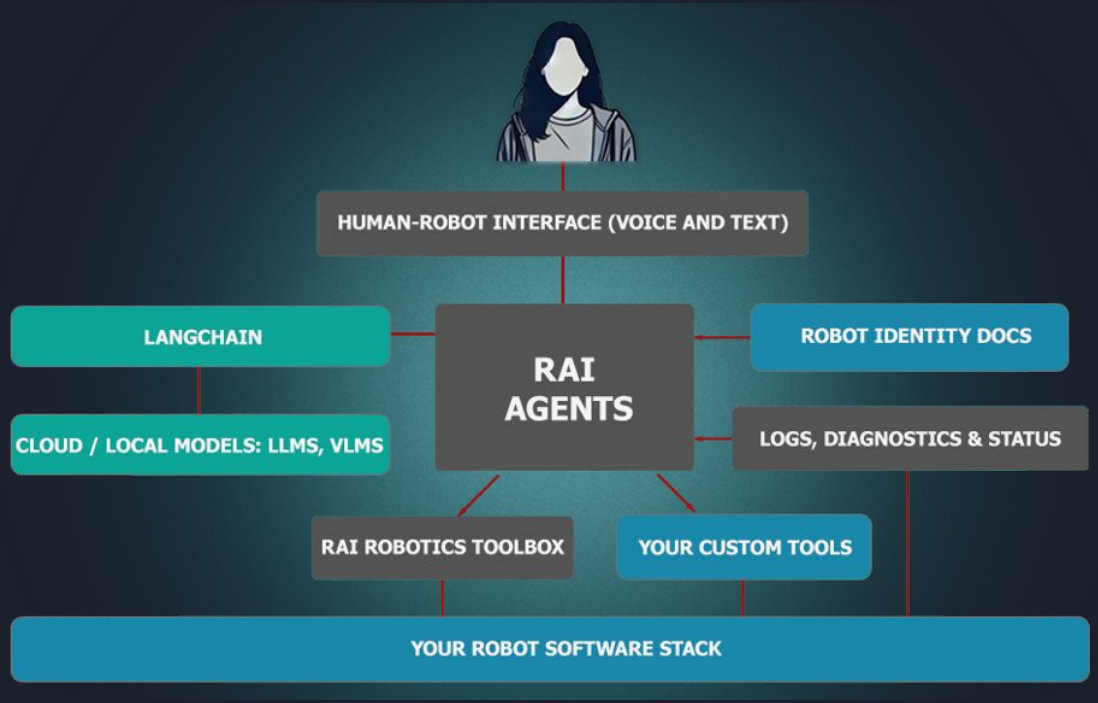
Introduction to ROS2: examples

Sample command:

```
ros2 topic pub /chatter std_msgs/msg/String "{data: 'Hello ROS2'}"
```



RAI System Architecture



LLMs: Langchain and Langraph integration

- Large Language Models and Vision Language Models conceptual wrapper
- Allows for integration with multiple LLMs and VLMs
- Provides functionality for multi-LLM based workflows
- Native capabilities of:
  - Tool calling
  - Logging
  - Tracing

RAI Connectors

RAI Connectors

- Provide an abstraction for communication with external APIs
- Support following "modes of communication":
  - `read_message`
  - `send_message`
  - `service_call`
  - `start_action`
  - `create_service`
  - `create_action`
- Inspired by ROS2, provide a general way of perceiving and actuating the environment, as well as communication between agents

- Def: a tool is any function with a described input
- Examples:
  - `divide(a: float, b: float) -> float //function`
  - `divide(a,b); where a is the numerator, b is the denumerator and the return value is result of division // tool`
- In practice most tools are impure functions – they have side effects

```python
class ReceiveROS2MessageToolInput(BaseModel):
    topic: str = Field(..., description="The topic to receive the message from")
    timeout_sec: float = Field(1.0, description="The timeout in seconds")


class ReceiveROS2MessageTool(BaseROS2Tool):
    connector: ROS2ARIConnector
    name: str = "receive_ros2_message"
    description: str = "Receive a message from a ROS2 topic"
    args_schema: Type[ReceiveROS2MessageToolInput] = ReceiveROS2MessageToolInput

    def _run(self, topic: str, timeout_sec: float = 1.0) -> str:
        if not self.is_readable(topic):
            raise ValueError(f"Topic {topic} is not readable")
        message = self.connector.receive_message(topic, timeout_sec=timeout_sec)
        return str({"payload": message.payload, "metadata": message.metadata})
```

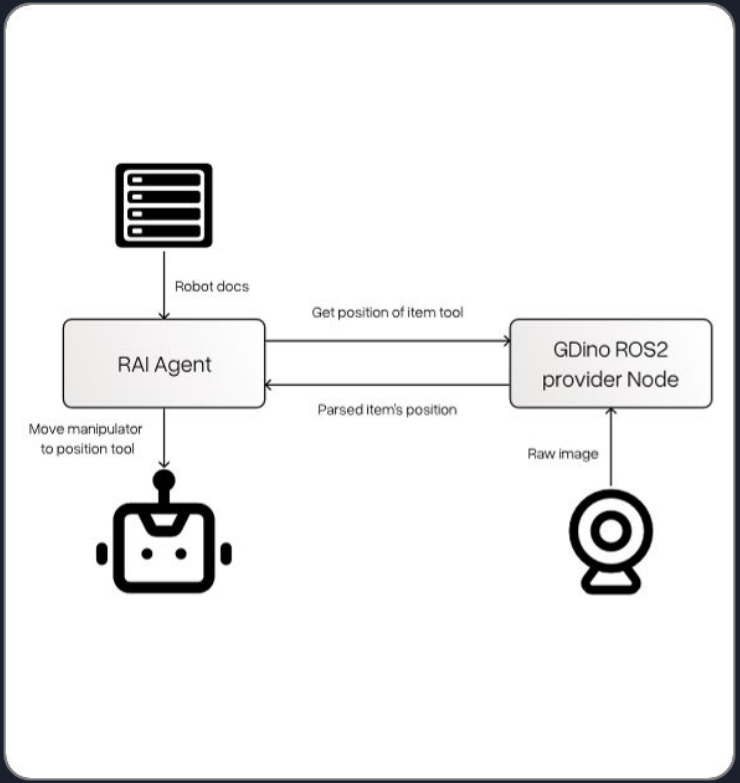
RAI Agents

- In RAI an Agent has:
  - Connectors – to interact with its environment
  - Tools - (usually) instructions on how to use these connectors
  - A reasoning process – either non-deterministic like LLM or a rule based one
- Examples of RAI Agents:
  - AutomaticSpeechRecognition Agent
  - TextToSpeech Agent
  - ReAct Agent
  - StateBased Agent

Use Case Walkthrough: Manipulation

Use Case Walkthrough: Manipulation

- Tools:
  - Get Object position
  - Move the manipulator
- Tasks:
  - Sorting
  - Stacking
  - Replacement



Use Case Walkthrough: Agriculture

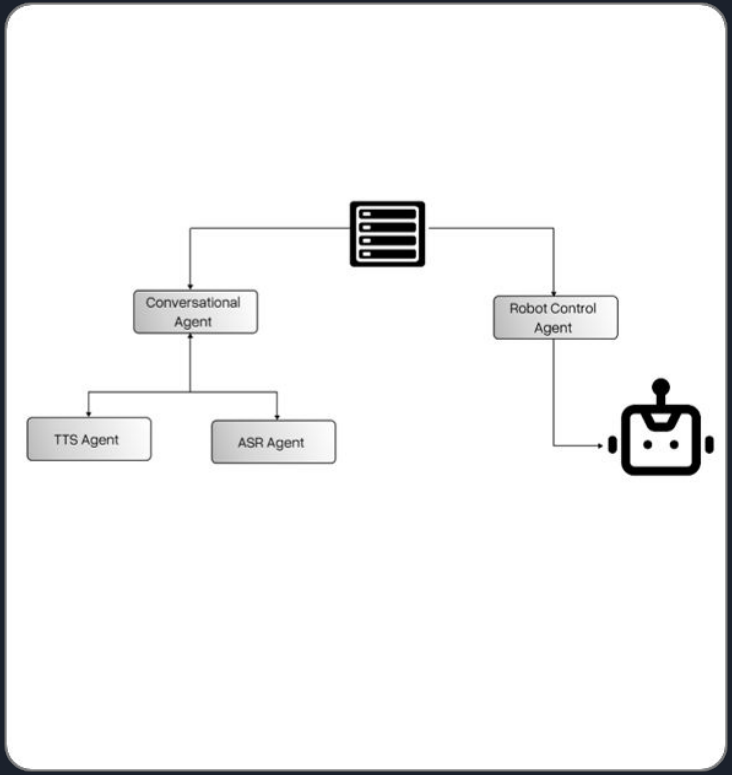
Use Case Walkthrough: Agriculture

- Tools:
  - Vision
  - ROS Service Calls (replan, move, honk)
- Handling unexpected situations
  - Small obstacles
  - Big obstacles
  - Cat 🐱
- Embodiment
  - Text
  - Text + image

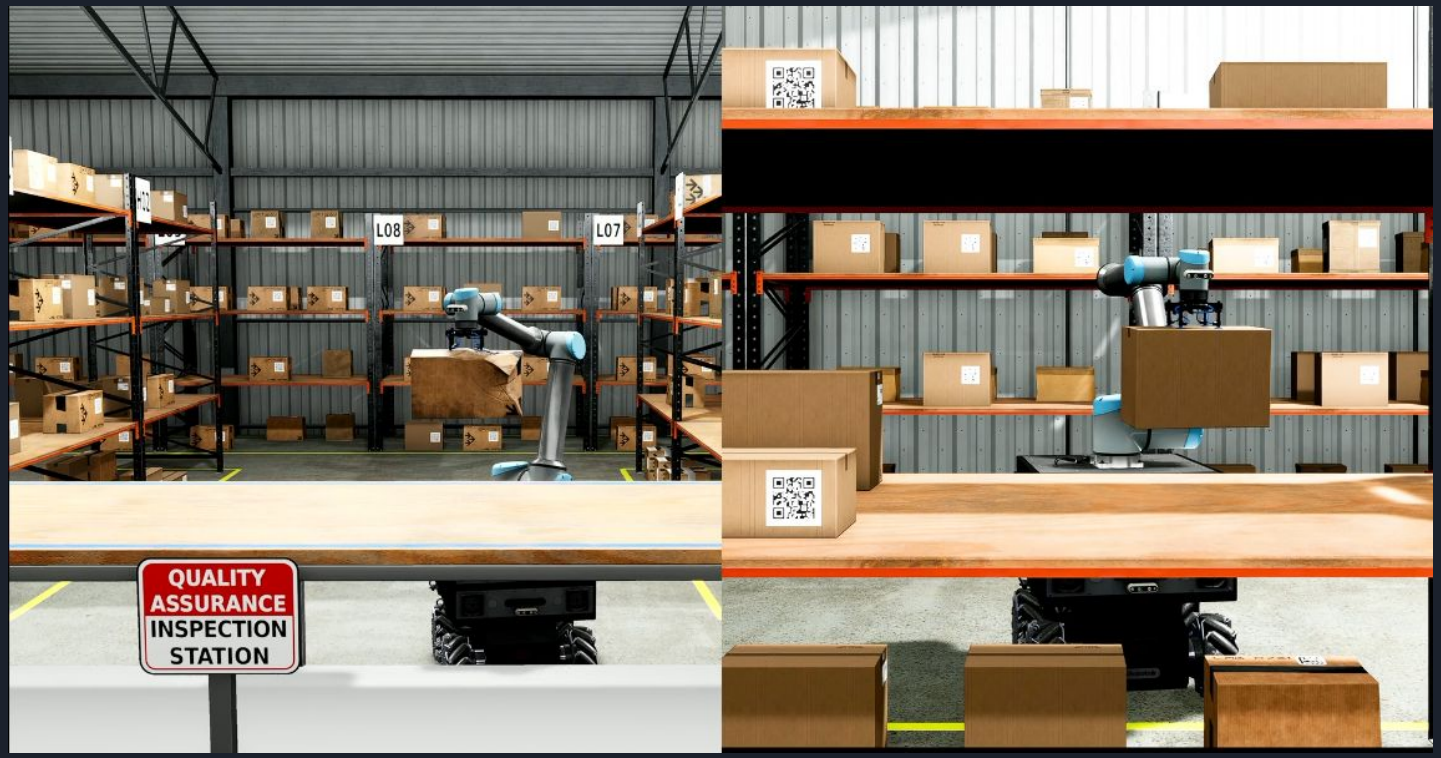
Use Case Walkthrough: ROSBotXL Husarion

Use Case Walkthrough: ROSBotXL Husarion

- Sensors
  - Camera
  - Depth Camera
  - Microphone
- Actuators
  - Speakers
  - Wheels
- Agents
  - Text to Speech
  - Automatic Speech Recognition
  - Conversational
  - Robot Control



Use Case Walkthrough: Mobile Manipulator



RAI in Multi-Agent Context

- Multi-agent Systems and Embodiment:
  - Single Embodiment - Multi Agent
  - Multi Embodiment - Multi Agent
- Single Embodiment:
  - Simple
  - Essentially multiple agents acting as one agent
- Multi embodiment
  - Complex

RAI in Multi-Agent Context

- **Challanges**
  - We don't know how well LLMs could reason about other robots in the environment
  - Negotiation strategies for task division, cooperation etc.
  - Rules enforcement

- **Interesting questions:**
  - What way should Agents communicate with each other?
  - Would a Multi-embodiment setup (with multiple autonomous agents) be more beneficial (in some way)?
  - Or should there be an orchestrator?

Future

- **Ethics!**
- Embodied AI vs World Model VLAs
- Feasible real world deployments

95% of LLM driven pilots fail - why?

Join the community

EMBODIED AI
COMMUNITY
GROUP

Join the community

- RAI is fully FOSS under Apache 2.0 License
- Source code available on GitHub RobotecAI/rai
- Simulation binaries presented in this demo available for download (so you can play around yourself)
- Contributors are welcome and will be guided

Embodied AI Discord invite

Contact

Kajetan Rachwał

Project Lead Machine Learning Engineer

kajetan.rachwal@robotec.ai